

RISK SENTINEL X

Evidence-Driven AI Risk Investigation & Governance Platform

MASTER DOCUMENTATION - REVIEWED, CORRECTED & ENHANCED

RiskSentinel X - Authoritative Runtime Decision Flow



Safety invariant: graph context is evaluated independently; the LLM is called only when the deterministic investigation

Low ML + low graph can bypass the LLM for latency/cost control. Post-decision workflows never overwrite the original decision.

Current engineering status

The architecture and documentation are mature, but the executable MVP is still being recovered. Database persistence has been implemented but runtime verification is pending; the IEEE-CIS training pipeline has been implemented but the dataset and generated model artifacts are still required; graph intelligence is the next recovery step.

Document	Master Engineering Specification
Project	RiskSentinel X
Track	AI Risk Manager / payment-risk investigation prototype
Version	Master Documentation v1.1 - Companion PDF Review Integrated
Date	27 August 2026

Defense-only research prototype. Uses public/synthetic data only; it does not use Razorpay production customer data.

Document Map

Use this section as the working table of contents.

Section	Title
1	Executive Summary
2	Current Repository Reality & Recovery State
3	Problem Statement
4	Vision, Objectives & Non-Goals
5	Users, Use Cases & Success Criteria
6	Target Architecture
7	Data Architecture & Persistence
8	ML Risk Model
9	Graph Intelligence
10	Controlled Investigation Agent
11	Deterministic Policy Engine
12	Auditability & Explainability
13	API & Backend Design
14	Razorpay-Inspired UI/UX Blueprint
15	Test Data Acquisition & Generation
16	Complete Testing Strategy
17	Evaluation & Metrics
18	Security, Safety & Failure Handling
19	Deployment & Local Environment
20	Recovery Roadmap & Implementation Order
21	Demo & Judging Runbook
22	Limitations, Claims & Ethics
23	Release Criteria & Definition of Done
24	Appendices: Payloads, Commands, Checklists & References
25	Companion PDF Review & Incorporated Enhancements

1. Executive Summary

What RiskSentinel X is, what it solves, and why the architecture is intentionally governed.

RiskSentinel X is an evidence-driven payment-risk investigation prototype designed to combine several complementary techniques instead of asking a single model or a large language model to make an opaque decision. Its core proposition is simple: models detect suspicious patterns, graphs connect related entities, a bounded AI agent investigates trusted evidence, and deterministic policy makes the final ALLOW, REVIEW, or BLOCK decision.

The problem is not merely to assign a fraud probability. A practical risk-operations workflow also needs to show why a case is suspicious, whether apparently separate transactions are connected, what evidence was consulted, which policy rule produced the final action, and whether the decision can be reconstructed later. RiskSentinel X therefore treats explainability, auditability, graceful degradation, and deterministic decision authority as first-class engineering requirements.

The target MVP is intentionally narrower than an enterprise fraud platform. It uses the public IEEE-CIS Fraud Detection benchmark for the tabular machine-learning component and a separately generated synthetic relationship graph for graph intelligence. The two evaluation domains are kept distinct. The project never claims that synthetic graph results represent Razorpay's real payment network, and it does not use Razorpay production data.

A strict repository audit found that the documentation had moved far ahead of executable code. The recovery plan in this document corrects that by prioritising a genuine working pipeline before any advanced v2 research. The shortest credible route is: persistence, real ML artifacts, real synthetic graph data, real bounded LLM integration, genuine end-to-end orchestration, a dashboard wired to persisted results, real held-out evaluation, and a final MVP validation gate.

Core product line

Models detect. Graphs connect. Agents investigate. Policies decide.

1.1 Target Outcome

- A transaction can enter one real API path and be persisted, scored by the trained model, checked against graph context, investigated using trusted tools, decided by deterministic policy, and written to an audit trail.
- The dashboard renders only backend results. It never calculates risk or final decisions in the browser.
- Every formal metric is calculated from real predictions and a defined evaluation dataset. Hardcoded metric JSON is prohibited.
- The LLM is advisory. High LLM confidence alone can never authorize a financial action.
- Failure of the LLM must degrade safely rather than crash the ML/graph/policy pipeline.

2. Current Repository Reality & Recovery State

The starting point for implementation; this section deliberately separates implemented code from verified runtime behavior.

Area	Current Status	Evidence / Meaning
Database	Implemented; runtime verification pending	SQLAlchemy/PostgreSQL foundation and persistent audit code have been added; local pytest and Docker execution were not available in the coding-agent shell.
ML training pipeline	Implemented; dataset required	A genuine XGBoost training pipeline exists, old fake evaluation files were quarantined, but IEEE-CIS files are absent so no artifacts or metrics exist yet.
RiskModelService	Securely blocked without artifacts	Inference refuses to score when the model/preprocessor are not present; this is correct behavior.
Graph intelligence	Next recovery task	Algorithms partially exist but the synthetic graph benchmark has not yet been generated and verified.
Agent	Partial	Provider/tool scaffolding exists, but normal execution still relies on a mock provider until recovery step 4.
Policy integration	Partial / mocked orchestration	Existing service historically used hash-derived ML/graph state and must later be rewired to real services.
Dashboard	Partial	UI exists but runtime data remains mock-backed until recovery step 6.
Formal evaluation	Not valid yet	Previously hardcoded metrics were renamed STALE_; genuine held-out evaluation is deferred until a trained model exists.
v2 research	Deferred	Calibration, counterfactuals, drift, retraining and release-governance research must wait until the core MVP passes the recovery gate.

Recovery Roadmap - Build the Real MVP Before v2 Research



Gate rule: Do not resume calibration, counterfactuals, drift, or retraining research until Step 8 proves the core pipeline works without runtime mocks.

Current implementation truth: a strong design exists, but a genuine end-to-end MVP is still under recovery.

Figure 2.1 - Recovery sequence based on the audited codebase.

2.1 Engineering Truth Rules

- Implemented but not executed is not the same as verified working.
- A written unit test is not counted as passed until the test runner executes it successfully.
- A Docker Compose file is not counted as a working deployment until services actually start and health checks pass.
- A model-training script is not a model artifact. The project remains blocked until real data produces a persisted model and validation metrics.
- Documentation, mock UI data, deterministic mock providers, sample JSON, and planned interfaces do not count as completed runtime features.

3. Problem Statement

Why risk scoring alone is insufficient.

Modern payment-risk systems must reason about more than a single transaction. A transaction may appear ordinary in isolation while belonging to a suspicious cluster that shares devices, IP addresses, payment instruments, merchants, or other contextual signals. Conversely, legitimate users can share IP addresses or devices, so simplistic linkage rules create false positives.

A second difficulty is operational explainability. Investigators need a compact explanation of what was observed, what historical context was checked, which graph links mattered, and why the final action was selected. A raw model score is useful but insufficient for review operations.

A third difficulty is safe AI integration. Large language models are good at summarising heterogeneous evidence but are non-deterministic and can produce unsupported statements. Giving an LLM direct authority to block or allow payments would create an unacceptable control boundary for this prototype. RiskSentinel X therefore uses AI only for bounded evidence synthesis and recommendation, while deterministic policy retains final authority.

Finally, risk decisions must be reproducible. If a case is reviewed later, the system should be able to recover the transaction, ML score, graph signals, tool calls, validated evidence, AI recommendation, policy rule and final decision. This is the basis for auditability and future governance.

3.1 Problems to Solve

Problem	Consequence	RiskSentinel X Response
Isolated transaction scoring	Missed coordinated behavior	Graph context links customers, devices, IPs and other entities.
Opaque score	Slow/manual investigation	Expose ML risk, graph signals, evidence and reason codes.
LLM hallucination risk	Unsupported or unsafe decisions	Bounded tools + evidence validation + deterministic final policy.
False positives from simple sharing	Legitimate users are over-flagged	Synthetic benchmark includes benign household/office sharing patterns.
No decision lineage	Hard to debug or defend outcomes	Append-oriented audit events and persisted decision records.
Metrics without provenance	Misleading engineering claims	Only computed predictions may generate official evaluation artifacts.

4. Vision, Objectives & Non-Goals

The product boundaries keep the MVP credible and testable.

4.1 Vision

Build a transparent risk-investigation system in which multiple signals are combined safely, AI is constrained to an advisory role, and every final action is explainable from recorded evidence and deterministic policy.

4.2 Primary Objectives

- Produce a genuine ML risk probability using a trained XGBoost model on public IEEE-CIS data.
- Detect suspicious relationship structure in a separate seeded synthetic graph without access to planted ground-truth labels.
- Use exactly bounded investigation tools to retrieve transaction history and graph context.
- Require structured AI output containing recommendation, confidence, reason codes and evidence.
- Use deterministic policy to issue ALLOW, REVIEW or BLOCK.
- Persist a complete audit trail and expose it in an operational dashboard.
- Evaluate ML and graph components honestly on their own data domains.
- Demonstrate graceful operation when the LLM provider is unavailable.

4.3 Non-Goals for the MVP

- No claim of Razorpay-scale throughput or production readiness.
- No use of Razorpay customer or merchant production data.
- No autonomous online retraining.
- No direct LLM enforcement authority.
- No production-grade distributed graph database; NetworkX is acceptable for prototype scale.
- No claim that IEEE-CIS anonymized variables represent particular real-world concepts unless documentation supports it.
- No attempt to recreate the Razorpay product UI pixel-for-pixel or use Razorpay trademarks as RiskSentinel branding.

5. Users, Use Cases & Success Criteria

Who uses the system and what a successful workflow looks like.

5.1 Primary Personas

Persona	Need	Key Screens
Risk Analyst	Quickly understand suspicious cases and evidence	Transactions, Investigation Detail, Review Queue, Audit
ML / Risk Engineer	Validate model behavior and integration	Model diagnostics, score endpoint, evaluation artifacts
Demo / Judge	Understand value in minutes	Overview, suspicious simulation, decision trace
Maintainer	Run services, inspect failures, reproduce cases	Health, logs, audit, setup guide

5.2 Core Use Cases

1. Process a normal transaction and show a low-risk decision with evidence lineage.
2. Process a transaction linked to a planted suspicious graph cluster and show multi-signal corroboration.
3. Demonstrate a case where the agent recommends BLOCK but weak graph corroboration forces the deterministic policy to REVIEW.
4. Disable the LLM provider and demonstrate that ML, graph, policy and audit continue in degraded mode.
5. Open a historical case and inspect the recorded sequence of scoring, graph checks, tool calls, recommendation and policy decision.

5.3 MVP Success Criteria

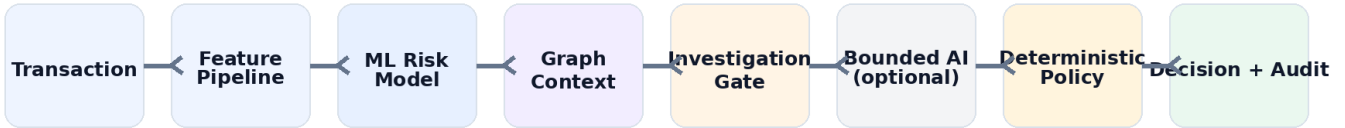
MVP completion gate

The project is not complete until a real transaction can traverse ML -> Graph -> Agent -> Policy -> Audit without runtime mock scores or hardcoded decisions.

6. Target Architecture

Authoritative runtime path and separation of responsibilities.

RiskSentinel X - Authoritative Runtime Decision Flow



Safety invariant: graph context is evaluated independently; the LLM is called only when the deterministic investic

Low ML + low graph can bypass the LLM for latency/cost control. Post-decision workflows never overwrite the original decision.

Figure 6.1 - Target authoritative runtime flow.

6.1 Component Responsibilities

Component	Responsibility	Must Not Do
FastAPI API	Validate input, call orchestration service, expose results	Compute risk in route handlers
RiskModelService	Load real model/preprocessor and produce probability	Use hash/random fallback
GraphRiskService	Compute graph signals and risk from real generated graph	Read planted ground-truth labels
InvestigationService	Call bounded tools and validate structured LLM output	Invent unverified evidence
PolicyService	Apply deterministic rules to trusted inputs	Call LLM for the final action
AuditService	Persist append-oriented event trail	Overwrite historical events
Next.js UI	Render backend truth and provide navigation/filtering	Calculate final decisions client-side

6.2 Authoritative vs Post-Decision Paths

The authoritative path ends when deterministic policy emits ALLOW, REVIEW or BLOCK and the result is persisted. Analyst feedback, model-health monitoring, future counterfactual research and retraining orchestration are post-decision workflows. They may create additional records but cannot rewrite the original decision.

This separation is the central safety property of the architecture. A future model challenger can be evaluated in shadow mode, but its score cannot enter PolicyService unless it has been explicitly promoted through a governed process.

7. Data Architecture & Persistence

Real persistence is the first foundation of the recovery.

7.1 Runtime Database

PostgreSQL is the intended runtime database, accessed through SQLAlchemy 2.0 with the psycopg driver. SQLite may be used for isolated tests where types are handled compatibly. For MVP bootstrapping, schema creation through `Base.metadata.create_all()` is acceptable, but it must be described honestly as schema initialization rather than a full migration history. Alembic should be added before production-oriented schema evolution is claimed.

Table	Purpose	Core Fields
transactions	Raw/normalized transaction record	transaction_id, customer_id, amount, timestamp, payload
risk_scores	Persist model results	transaction_id, model_version, score
investigations	Persist agent result/state	transaction_id, status, recommendation, confidence, reason_codes
decisions	Persist final policy action	transaction_id, final_decision, policy_version, reason
audit_events	Append-oriented event history	transaction_id, timestamp, event_type, component, payload

7.2 Data Integrity Rules

- Transaction IDs should be idempotent at the orchestration boundary.
- Completed decisions must have an audit record; the UI must never show a completed case whose audit persistence failed.
- Model/graph/agent/policy outputs should be persisted as separate concepts rather than collapsed into one “risk” field.
- Audit payloads may include structured JSON but should avoid secrets and hidden reasoning traces.
- Database health must be checked using a real query such as `SELECT 1`.

7.3 Persistence Tests

- Insert/read transaction
- Persist/read risk score
- Append and order audit events
- Persist decision
- Verify rollback on database error
- Health endpoint returns degraded/unhealthy when DB is unavailable

8. ML Risk Model

Real IEEE-CIS training, strict split hygiene and reproducible inference.

8.1 Data Source

The tabular ML component uses the IEEE-CIS Fraud Detection competition dataset hosted on Kaggle. The official dataset separates transaction and identity data and joins them with TransactionID; the binary target is isFraud. The training files are train_transaction.csv and train_identity.csv. The project should not use Kaggle's unlabeled test files for internal supervised evaluation because the target is unavailable there.

8.2 File Placement

```
data/raw/
  train_transaction.csv
  train_identity.csv
```

- Do not commit raw competition data to Git.
- Accept the Kaggle competition rules before downloading.
- Record dataset presence and hashes locally where practical.

8.3 Recommended Split

Sort labeled training transactions by TransactionDT and use a deterministic chronological 70% train / 15% validation / 15% held-out test split. TransactionDT is a timedelta from a reference point rather than an absolute timestamp, but it still supports ordering. Fit preprocessing only on the train partition. Use validation for early stopping, threshold selection and any limited model selection. Do not inspect held-out test labels until the final evaluation step.

Data and Evaluation Separation

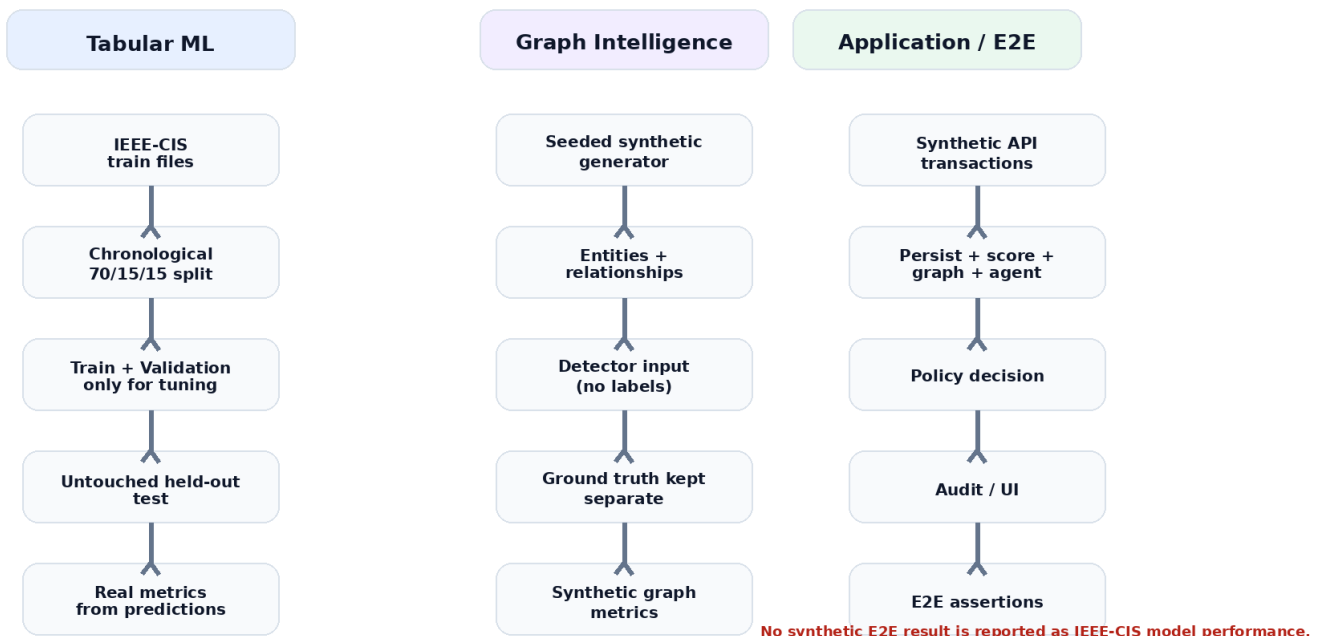


Figure 8.1 - Separation of ML, graph and application test data.

8.4 Preprocessing

- Exclude isFraud from predictors and treat TransactionID as an identifier.

- Handle numeric missing values deterministically.
- Handle categorical missing and unseen categories using a persisted encoder or deterministic mapping.
- Preserve exact feature order in a feature manifest.
- Do not invent semantics for anonymized V*, C*, D* features.

8.5 XGBoost Training

A practical initial configuration is binary:logistic with aucpr evaluation, moderate depth, controlled learning rate, subsampling, column subsampling and scale_pos_weight computed from train class counts. The objective is not to reproduce a leaderboard solution. The objective is to produce a real, reproducible, inspectable baseline suitable for the integrated risk pipeline.

8.6 Required Artifacts

```
models/
  xgb-ieeeecis-v1.json
  preprocessor-v1.joblib
  feature_manifest-v1.json
  threshold-v1.json
  model_metadata-v1.json
```

8.7 Validation Metrics

- Average Precision (AP)
- Trapezoidal PR-AUC if separately computed and clearly labeled
- Precision
- Recall
- F1
- TP, FP, TN, FN
- Validation-selected threshold

Do not confuse AP and PR-AUC

Average Precision and area under the precision-recall curve can be computed differently. If both are reported, label the implementation used for each.

8.8 Current ML Recovery Status

The training pipeline has been implemented and legacy hardcoded evaluation files were renamed with a STALE_ prefix. The IEEE-CIS files are not yet present in data/raw, therefore no model artifacts, validation metrics, or real /score execution can be claimed. Once the dataset is supplied, the training script must be executed and the resulting artifacts verified before integration proceeds.

9. Graph Intelligence

Separate synthetic relationship benchmark with planted ground truth kept away from the detector.

9.1 Why Graphs

Tabular ML evaluates one transaction's features, while graph analysis exposes relationships across entities. Coordinated abuse frequently manifests through reuse or dense connectivity, but legitimate households and workplaces also share infrastructure. The graph benchmark must therefore include both suspicious clusters and benign sharing.

9.2 Synthetic Entities

- CUSTOMER
- DEVICE
- IP
- MERCHANT where useful
- PAYMENT_INSTRUMENT where useful
- TRANSACTION nodes or relationship edges depending on the existing implementation

9.3 Generator Outputs

```
data/synthetic/
  entities.csv
  relationships.csv
  ground_truth.csv
  graph_manifest_v1.json
```

Ground-truth isolation

GraphRiskService must work when ground_truth.csv is absent. Only the evaluation script may join detector predictions to planted labels.

9.4 Legitimate Patterns

- Household shared IP
- Family shared device
- Office/public NAT-like IP sharing
- Many customers paying a common merchant
- Occasional device reuse without dense multi-signal linkage

9.5 Suspicious Patterns

- Multiple customers sharing several devices
- Multiple customers sharing the same IP plus another entity
- Repeated payment-instrument reuse
- Dense multi-entity communities
- Multiple independent graph signals appearing together

9.6 Graph Signals & Risk

The detector should calculate only signals that exist in the actual graph representation: connected customer count, shared device/IP/payment-instrument counts, community size/density and similar structural features. A deterministic weighted formula can convert normalized signals into graph_risk in [0,1]. The formula and threshold should be versioned. It must never reference planted labels.

9.7 Graph Evaluation

- Customer-level or clearly defined entity-level evaluation
- Precision, Recall, F1, TP, FP, TN, FN
- Optional PR-AUC for a continuous graph score
- Seed, graph version, threshold and evaluation unit recorded
- Explicit label: SYNTHETIC GRAPH BENCHMARK

9.8 Detector Community IDs vs Planted Ground Truth

The graph detector must return its own detected community identifier (for example COMM-17), never the planted benchmark identifier (for example SC-03). The planted cluster ID exists only in ground_truth.csv and evaluation artifacts. Returning SC-03 from /graph-check would leak the answer into runtime and invalidate the graph benchmark.

Leakage guard

Normal graph responses may include detected_community_id, graph_risk, connected_entities and structural signals. They must never expose ground_truth_label or planted_cluster_id.

9.9 Avoid Label-Derived Graph Features

Any signal named suspicious-neighbor ratio is unsafe if it means “neighbors whose planted ground-truth label is suspicious.” For the MVP either omit that signal or define it only from detector-produced structural risk computed without labels. Ground-truth labels may be joined only after all detector predictions are frozen for evaluation.

10. Controlled Investigation Agent

Use an LLM to synthesize evidence, never to create enforcement authority.

10.1 Exactly Bounded Tools

```
get_transaction_history(customer_id)
get_graph_context(entity_id)
```

These tools should query real persisted transaction history and real graph context. The agent is not allowed to browse arbitrary databases or invent evidence. The provider abstraction may support Anthropic/Claude and a local Ollama provider when configured, while MockProvider remains test-only.

10.2 Deterministic Investigation Gate

To control latency and provider cost without sacrificing graph detection, ML and graph checks should complete before the LLM is considered. A small deterministic investigation gate decides whether the agent is needed. Recommended MVP routing: ML low + graph low -> skip agent and let policy evaluate trusted non-LLM signals; ML medium/high OR graph medium/high -> call the agent. This gate decides whether to investigate, not whether to ALLOW/BLOCK.

```
ML score + graph risk
    |
    v
InvestigationGate v1
  LOW + LOW  -> SKIP_AGENT
  otherwise -> RUN_AGENT
    |
    v
Deterministic Policy (final authority)
```

- Version the gate thresholds/configuration.
- Record gate outcome in audit: SKIP_AGENT or RUN_AGENT.
- A low ML score must not suppress a strong graph anomaly.
- Skipped-agent cases must not fabricate an agent recommendation.

10.3 Structured Output

```
{
  "recommendation": "ALLOW | REVIEW | BLOCK",
  "confidence": 0.0,
  "reason_codes": [...],
  "evidence": [
    {"source": "transaction_history | graph_context", "fact": "..."}
  ]
}
```

10.4 Evidence Validation

- Every fact in evidence must map to a tool result.
- Unsupported evidence is rejected before PolicyService sees it.
- Reason codes must belong to an allowed schema.
- The agent may be retried a bounded number of times for invalid JSON; repeated failure produces DEGRADED state.
- Transaction text and analyst comments are treated as untrusted data, not instructions.

10.5 Provider Strategy & Failure Handling

Provider choice must remain swappable and non-authoritative. Use a hosted provider when configured and keep Ollama as the local fallback. OpenRouter currently lists stealth/ox-alpha as a free preview with a

1,048,576-token context window and tool-calling support, but preview availability can change. Do not publish unverified latency/error statistics or speculate about the model's hidden developer/family. Provider failure must return a structured DEGRADED result; deterministic policy continues using available ML/graph evidence.

11. Deterministic Policy Engine

The single component with authority to emit the final action.

11.1 Inputs

- Real ML risk result
- Real graph risk/signals
- Validated agent recommendation and evidence
- Agent execution state such as COMPLETED or DEGRADED

11.2 Outputs

- ALLOW
- REVIEW
- BLOCK
- Triggered rule IDs / reason
- Policy version

11.3 Critical Safety Invariant

Mandatory regression

Agent recommendation = BLOCK and confidence = 0.99 with weak/insufficient graph evidence must NOT automatically produce BLOCK. High AI confidence alone is never enough.

11.4 Policy Design Example

Exact thresholds must come from validation and be stored in configuration rather than invented in UI code. A reasonable structural pattern is: low ML risk with no strong graph signal may ALLOW; uncertain or conflicting evidence should REVIEW; BLOCK should require multiple verified conditions such as high ML risk plus strong graph corroboration and valid evidence. The project must test threshold boundaries just below, exactly at, and just above each configured threshold.

11.5 Remove Legacy Mock Integration

The audited PolicyService historically used a hash of transaction_id to simulate ML and graph state. That logic must be removed from the normal runtime path. PolicyService should receive already-computed trusted inputs, while a separate orchestration service coordinates the real sequence.

12. Auditability & Explainability

Every completed case should be reconstructable without exposing hidden chain-of-thought.

12.1 Minimum Audit Fields

- transaction_id and case_id
- timestamp
- ML score and model version
- graph risk/signals and graph version
- tool calls and result status
- validated evidence
- reason codes
- agent recommendation/confidence/state
- policy version and triggered rules
- final decision

12.2 What Must Not Be Logged

- API secrets
- Provider credentials
- Hidden chain-of-thought
- Raw sensitive values that are unnecessary for investigation
- Unsupported model-generated claims

12.3 Explainability Model

RiskSentinel X exposes operational evidence rather than private reasoning. The analyst sees score, graph context, tool calls, reason codes and structured recommendation. The goal is to answer “what evidence and rules produced this decision?” without presenting a hidden reasoning transcript.

13. API & Backend Design

Minimal endpoints that map cleanly to service responsibilities.

Endpoint	Purpose	Source of Truth
GET /health	Backend + DB/service health	Live service checks
POST /score	Diagnostic/model scoring	RiskModelService
GET /graph-check/{entity_id}	Graph context/risk	GraphRiskService
POST /investigate	Bounded investigation	InvestigationService
POST /decision or /process	End-to-end authoritative flow	Orchestration + PolicyService
GET /transactions	Persisted transaction list	Database
GET /cases/{id}	Full investigation detail	Database/audit
GET /events or SSE	Real-time case lifecycle	Backend event stream

13.1 Orchestration Service

```

validate request
-> persist transaction
-> RiskModelService.score(payload)
-> GraphRiskService.check_entity(...)
-> InvestigationService.investigate(context)
-> PolicyService.evaluate(trusted_results)
-> persist decision
-> append audit events
-> publish UI event

```

13.2 Error Contracts

- Validation error: 4xx structured schema error
- Missing model artifact: service unavailable/configuration error, never a fake score
- Graph data missing: explicit graph initialization error
- LLM provider unavailable: investigation DEGRADED, not entire service crash
- Database unavailable: no false success response
- Unexpected server errors: sanitized response; stack trace only in internal logs

14. Razorpay-Inspired UI/UX Blueprint

Original RiskSentinel design using payment-operations patterns: overview, transaction list, filters, drill-down details and actionable status.

Razorpay's public dashboard documentation describes a central operations dashboard with transaction activity, analytics, recent actions, time filters and drill-down payment details. RiskSentinel X should adapt those interaction patterns because they are familiar for payment operations: a clear navigation system, high-density transaction tables, filters, status chips, summary cards and a detail panel. The implementation should remain visually original and must not present itself as an official Razorpay product.

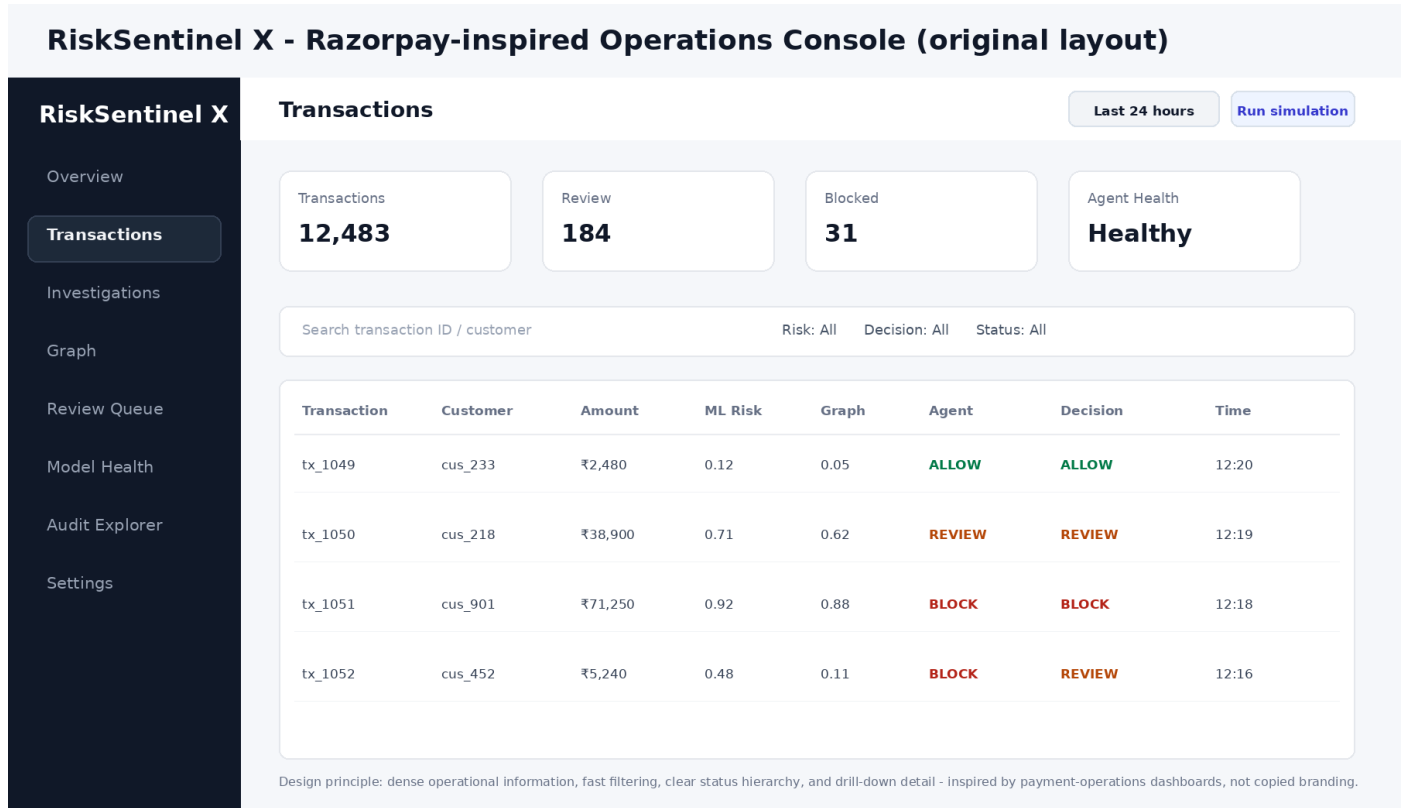


Figure 14.1 - Original RiskSentinel wireframe inspired by payment-operations dashboard patterns.

14.1 Information Architecture

- Overview
- Transactions
- Investigations
- Graph
- Review Queue
- Model Health (later / admin)
- Audit Explorer
- Settings / Health

14.2 Visual Language

Element	Specification
Navigation	Deep neutral/navy rail or compact side navigation; current section strongly highlighted.
Canvas	Light neutral background with white cards and subtle borders.
Typography	Inter/Aptos/Liberation Sans style; compact 14-16 px body,

	strong numeric KPI hierarchy.
Primary Accent	Cool blue/indigo accent; do not copy Razorpay trademark artwork.
Risk Status	Green = ALLOW/low, amber = REVIEW/uncertain, red = BLOCK/high; always pair color with text/icon.
Tables	Sticky header, search, filters, pagination, sortable time/risk columns.
Detail	Right-side panel or full case page with explicit ML / Graph / Agent / Policy sections.
Charts	Use only where they answer an operations question: volume, risk distribution, review rate, health.

14.3 Overview Screen

- KPI cards: transactions processed, review queue, blocked count, agent health.
- Recent activity table.
- Risk distribution chart or decision distribution.
- System health banner only when action is needed.
- Primary demo action: Simulate Suspicious Collusion Pattern.

14.4 Transactions Screen

- Search by transaction/customer ID.
- Filters: date/time, ML risk band, graph risk band, decision, investigation state.
- Columns: transaction ID, customer, amount, ML risk, graph risk, advisory recommendation, final decision, timestamp.
- Click row to open detail; do not overload the table with full evidence.

14.5 Investigation Detail

- Header: transaction ID, final decision chip, timestamp.
- Transaction summary.
- ML card: score, model version, top available model metadata (avoid inventing semantic explanations for anonymized features).
- Graph card: community, connected entities, signals/reason codes.
- Agent card: tools called, evidence, advisory recommendation, confidence, provider/degraded state.
- Policy card: final action, rule IDs, policy version.
- Audit timeline: chronological event list.

14.6 Review Queue

If human review is implemented after the MVP, the queue should show only persisted cases. Analyst assessment is a separate post-decision record and must never visually replace the original policy decision.

14.7 Accessibility & UX Safety

- Never rely on color alone for risk state.
- Use concise evidence labels rather than long AI prose.
- Keep final policy decision visually more prominent than the AI recommendation.
- Show DEGRADED clearly without implying the case was fully investigated.
- Provide loading/empty/error states for every data panel.

14.8 Razorpay Interaction Patterns to Adapt (Not Copy)

Current public Razorpay dashboard documentation emphasizes a central operational overview, recent activity, time-range filters, transaction lists, status/date filters and click-through payment details. RiskSentinel X should reuse those interaction principles while keeping original branding and risk-specific content.

- Top-level overview with a selectable time range such as Last 24 hours / 7 days / 30 days.
- Recent activity / cases requiring attention.
- Transactions table with date/time and decision/status filters.
- Click a transaction ID to open a focused detail view or side drawer instead of navigating through unrelated screens.
- Quick action for the defensive simulation, visually separated from transaction decision controls.
- A compact “Critical actions” banner only for service degradation or unresolved review backlog.

14.9 Read-Only Policy & Agent Rules View

For judging and debugging, add an admin/read-only page that shows the active investigation-gate version, allowed agent tools, policy version and rule IDs. Do not let the demo UI edit thresholds or policy rules. This makes the system easier to explain without creating a dangerous “tune until it blocks” control surface.

15. Test Data Acquisition & Generation

Exactly where to obtain real benchmark data and how to create safe synthetic data.

15.1 IEEE-CIS Data - Official Source

Official source: Kaggle IEEE-CIS Fraud Detection competition. The data page provides train_transaction.csv, train_identity.csv, test_transaction.csv, test_identity.csv and sample_submission.csv, subject to the competition rules. RiskSentinel X needs the two labeled training files for its own internal split and evaluation.

Required local paths:

```
data/raw/train_transaction.csv
data/raw/train_identity.csv
```

15.2 Download Procedure

6. Create/sign in to a Kaggle account.
7. Open the IEEE-CIS Fraud Detection competition and accept any required competition rules.
8. Download the data manually from the Data tab, or use the Kaggle CLI after configuring your Kaggle credentials.
9. Extract/copy train_transaction.csv and train_identity.csv into data/raw/.
10. Confirm data/raw/ is excluded by .gitignore.
11. Run the training command only after the files are present.

```
# Example Kaggle CLI workflow (exact authentication can vary by Kaggle CLI version)
kaggle competitions download -c iieee-fraud-detection -p data/raw
# unzip the downloaded archive, then verify the two required train files exist
```

Competition data rules

Do not redistribute Kaggle competition data from the repository. Keep local acquisition instructions and honor the competition rules/license shown by Kaggle.

15.3 Synthetic Graph Data

Graph data should be generated by the project itself, not downloaded from an unknown fraud dataset. Use a seeded generator so every demo graph is reproducible. Store entity/relationship inputs separately from ground_truth.csv. The generated graph is safe for public demos because it contains no real users.

15.4 API / E2E Test Fixtures

```
{
  "transaction_id": "tx_demo_001",
  "customer_id": "cus_demo_001",
  "amount": 2480.00,
  "timestamp": "2026-08-27T12:00:00+05:30",
  "features": {
    "ProductCD": "W",
    "TransactionAmt": 2480.0
  }
}
```

The final real request schema depends on how the full IEEE-CIS feature contract is adapted to the application. Do not silently populate hundreds of missing benchmark features with fabricated values. Define an explicit model-compatible payload or adapter.

15.5 Simulation Fixture Compatibility

The suspicious-collusion simulator must not force an ML risk score. The ML model is trained on an IEEE-CIS-compatible feature schema, so every simulated transaction must either (a) provide a complete model-compatible feature dictionary generated from a documented fixture template, or (b) reference a sanitized benchmark-compatible feature fixture stored under simulation/fixtures/. Graph entity links are then attached separately. The simulator may choose fixtures known to exercise the frozen policy, but the model, graph and policy still compute their own outputs.

- Every simulation run receives simulation_session_id and deterministic seed.
- Persist simulation flag/session separately from normal cases.
- Never set risk_score, graph_risk or final_decision inside the fixture.
- Do not reuse planted ground-truth cluster IDs in runtime responses.
- A simulator result is an E2E test result, not IEEE-CIS held-out model performance.

15.6 Data Labels and Claims

Data	Purpose	Can be called real fraud ground truth?
IEEE-CIS labeled train data	Tabular ML train/validation/held-out evaluation	Yes, within the public benchmark context
Project synthetic graph	Graph detection benchmark	Only synthetic planted labels; never real customer fraud
Synthetic E2E transactions	Pipeline tests/demo	No; test fixtures only
Mock LLM response	Unit tests	No; provider fixture only

16. Complete Testing Strategy

Unit, integration, E2E, failure-mode and data-leakage tests.

16.1 Test Pyramid

- Unit tests: pure logic and schema validation.
- Service tests: database/model/graph/agent/policy components in isolation.
- Integration tests: real service boundaries and persistence.
- E2E tests: one transaction through the complete authoritative path.
- Manual demo tests: UI, provider outage, simulation, audit inspection.

16.2 Database Tests

ID	Scenario	Expected Result
DB-01	Insert/read transaction	Exact persisted transaction returned
DB-02	Append 3 audit events	All retained in chronological order
DB-03	Persist final decision	Decision references transaction
DB-04	DB unavailable	Health is degraded/unhealthy; no false success
DB-05	Rollback on failure	Partial write not committed

16.3 ML Tests

ID	Scenario	Expected Result
ML-01	Train/validation/test IDs	No overlap
ML-02	Target leakage scan	isFraud absent from predictors
ML-03	Unknown category	Handled by persisted preprocessing
ML-04	Missing model artifact	Clear failure; no fake score
ML-05	Prediction range	$0 \leq \text{probability} \leq 1$
ML-06	Feature schema mismatch	Request rejected
ML-07	Threshold provenance	Threshold references validation artifact

16.4 Graph Tests

ID	Scenario	Expected Result
GR-01	Same seed twice	Stable entities/edges/labels or stable hash
GR-02	Detector without ground_truth.csv	Still works
GR-03	Benign shared IP	Not automatically maximum risk
GR-04	Dense multi-signal planted cluster	Higher graph risk than benign fixture
GR-05	Unknown entity	Structured not-found/empty result
GR-06	Risk range	$0 \leq \text{graph_risk} \leq 1$

16.5 Agent Tests

ID	Scenario	Expected Result
----	----------	-----------------

AG-01	Valid provider JSON	Schema accepted
AG-02	Invalid JSON	Bounded retry or DEGRADED
AG-03	Unsupported evidence	Evidence rejected
AG-04	Provider timeout	DEGRADED result; pipeline continues
AG-05	Prompt injection in transaction/comment	Treated as data
AG-06	MockProvider in normal runtime	Disallowed unless explicit test mode

16.6 Policy Tests

ID	Scenario	Expected Result
PO-01	Low ML + weak graph	ALLOW according to frozen policy
PO-02	Uncertain/conflicting signals	REVIEW
PO-03	Strong verified multi-signal evidence	BLOCK
PO-04	Agent BLOCK 0.99 + weak graph	NOT automatic BLOCK
PO-05	Agent unavailable	Safe deterministic fallback
PO-06	Boundary values	Correct behavior below/at/above threshold

16.7 E2E Tests

ID	Flow	Pass Condition
E2E-01	Normal transaction	Persist -> ML -> graph -> agent -> policy -> audit -> UI
E2E-02	Suspicious synthetic cluster	Real graph signals visible; decision from policy
E2E-03	LLM outage	ML/graph/policy/audit survive
E2E-04	Duplicate transaction	Idempotent behavior; no contradictory duplicate decision
E2E-05	Malformed input	Validation error; nothing partially persisted
E2E-06	Audit replay	Recorded inputs reproduce deterministic policy decision

16.8 Test Execution Truth

Written != passed

A test file does not count as a passed test. Record the exact command, pass/fail/skip counts, environment and any reason a test could not run.

16.9 MVP Acceptance Matrix - 15 Mandatory Cases

#	Test Case	Expected Behavior
1	Valid transaction	Validated, persisted and routed through the real pipeline.
2	Missing / malformed fields	Structured validation error; no partial success.

3	Duplicate transaction ID	Idempotent reject/deduplicate/replay behavior, explicitly documented.
4	Low ML + low graph	ALLOW/low path according to policy; investigation gate may skip LLM.
5	High ML or graph anomaly	Agent investigation is triggered by the deterministic gate.
6	Planted synthetic cluster	Graph detector flags structural risk without reading ground truth.
7	Required agent tools	Both trusted tools execute when RUN_AGENT is selected.
8	Invalid agent JSON	Bounded retry or DEGRADED; unsafe output not accepted.
9	Agent omits/claims unsupported evidence	Evidence validation rejects it.
10	Policy ALLOW fixture	Final decision produced by PolicyService.
11	Policy REVIEW fixture	Ambiguous/conflicting evidence produces REVIEW.
12	Policy BLOCK fixture	Only strong verified multi-signal evidence can produce BLOCK.
13	Agent BLOCK + thin graph evidence	Policy overrides/downgrades; no automatic BLOCK.
14	LLM/provider unavailable	Agent DEGRADED; ML/graph/policy/audit continue safely.
15	Audit completeness	Every completed decision has a queryable evidence + rule trail.

17. Evaluation & Metrics

Formal evidence for the ML and graph components, kept separate.

17.1 ML Final Held-Out Evaluation

After the model, preprocessing and validation-derived threshold are frozen, run the untouched 15% held-out partition exactly once for the formal MVP evaluation. Load the persisted artifacts rather than retraining inside the evaluation script. Save predictions and compute metrics directly using scikit-learn or an equivalent library.

- Precision
- Recall
- F1
- Average Precision
- PR-AUC if separately defined
- Confusion matrix
- TP / FP / TN / FN
- Accuracy as a secondary metric
- Brier / Log Loss / ECE only if calibration is genuinely implemented

17.2 Graph Evaluation

Run detector predictions first, then load synthetic ground truth and join for evaluation. Report graph metrics as a “Synthetic Graph Benchmark” with the generator seed, graph-risk version, threshold and evaluation unit. Never merge graph F1 with IEEE-CIS model metrics into a single system accuracy.

17.3 Illustrative Cost

$$\text{IllustrativeCost} = \text{FP} * \text{FP_Cost} + \text{FN} * \text{FN_Cost}$$

This can help explain business tradeoffs, but the costs are illustrative assumptions and must never be described as Razorpay economics. The assumptions must be fixed before comparing alternatives.

17.4 Metric Integrity

- Every README number links to a machine-readable source artifact.
- Old hardcoded metric files remain quarantined or removed from authoritative documentation.
- No manual editing of final metrics JSON.
- Test split membership and threshold provenance are documented.

18. Security, Safety & Failure Handling

Controls that make the prototype defensible.

18.1 Security Baseline

- Secrets only in environment variables; never in source, screenshots or logs.
- Sanitized API errors; no raw stack traces returned to clients.
- Safe SQLAlchemy usage; no string-concatenated SQL from user input.
- CORS restricted appropriately for the local frontend.
- Analyst free text rendered safely; no raw HTML injection.
- Provider keys never sent to frontend.

18.2 AI Safety Boundaries

- LLM advisory only.
- Only trusted tools provide evidence.
- Unsupported claims rejected.
- No hidden chain-of-thought display.
- Provider outage degrades safely.
- Prompt-injection content treated as data.

18.3 Failure Matrix

Failure	Required Behavior
Model artifact missing	Fail model startup/scoring explicitly; no random fallback.
Graph files missing	Graph service error; never fabricate graph risk.
LLM unavailable	Agent DEGRADED; policy continues with available trusted inputs.
Invalid LLM JSON	Bounded retry then DEGRADED.
Database unavailable	No completed decision returned without persistence/audit guarantee.
SSE/WebSocket disconnected	Backend processing continues; frontend reconnects or refreshes via REST.
Duplicate transaction	Idempotent result or explicit safe conflict behavior.

19. Deployment & Local Environment

A reproducible local demo is more valuable than unverified infrastructure claims.

19.1 Intended Services

```
docker compose
  postgres
  backend (FastAPI)
  frontend (Next.js)
  optional local Ollama provider
```

19.2 Environment Variables

- DATABASE_URL
- MODEL_ARTIFACT_PATH or model registry directory
- GRAPH_DATA_DIR
- LLM_PROVIDER
- ANTHROPIC_API_KEY if used
- OLLAMA_BASE_URL if used
- frontend backend base URL

19.3 Startup Checklist

12. Copy .env.example to .env and fill only required local values.
13. Ensure IEEE-CIS data is present before training, not necessarily before inference if artifacts already exist.
14. Generate synthetic graph data.
15. Train and persist ML artifacts.
16. Start Docker services.
17. Verify /health reports backend and database status.
18. Process one diagnostic /score and /graph-check request.
19. Run one full process/decision request.
20. Open the dashboard and confirm persisted data appears.

19.4 Current Verification Gap

In the current recovery environment, the coding agent reported that Python and Docker CLI commands were unavailable, so database tests and Docker startup were not actually executed. These remain explicit verification tasks on the developer machine even though the code has been written.

19.5 Case Lifecycle & Real-Time Events

Use an explicit case lifecycle so the frontend can stream progress without guessing: RECEIVED -> PERSISTED -> SCORED -> GRAPH_CHECKED -> AGENT_SKIPPED or INVESTIGATING -> DECIDED -> AUDITED. Failure/degradation states should be explicit rather than inferred from missing fields.

```
case_id / transaction_id / correlation_id
event_type
event_timestamp
component
status
safe payload summary
```

SSE/WebSocket reconnects should request or reload the persisted case state so a temporary frontend disconnect never loses the authoritative history.

20. Recovery Roadmap & Implementation Order

The minimum route from the audited shell to a credible working MVP.

Recovery Roadmap - Build the Real MVP Before v2 Research



Gate rule: Do not resume calibration, counterfactuals, drift, or retraining research until Step 8 proves the core pipeline works without runtime mocks.

Current implementation truth: a strong design exists, but a genuine end-to-end MVP is still under recovery.

Step	Outcome	Gate to Move Forward
1. Database	Real persistence and audit	pytest DB tests + PostgreSQL/Docker health actually pass
2. ML	Real IEEE-CIS model artifacts	Dataset present, training executed, validation metrics real
3. Graph	Generated synthetic graph + evaluation	Detector runs without ground truth; real graph metrics
4. Agent	Real provider + real tool logic	Structured evidence validated; provider-failure tests pass
5. E2E integration	One real authoritative pipeline	No hash/random/mock scores in runtime
6. Dashboard	Persisted cases rendered	No MOCK_TRANSACTIONS in normal runtime
7. Evaluation	Formal held-out metrics	Predictions generated from frozen artifacts
8. MVP validation	Release gate	E2E, Docker, audit, safety and claims all pass

20.1 Why v2 Research Is Deferred

Calibration experiments, counterfactual explanations, model monitoring, analyst-feedback retraining and champion/challenger governance are valuable only after the core system is executable. Implementing them on top of mock scores would create more documentation without improving the credibility of the project. The correct gate is “GENUINE RISKSENTINEL X MVP COMPLETE.”

21. Demo & Judging Runbook

A six-minute story that demonstrates the architecture rather than a random feature tour.

21.1 Demo Sequence

Time	Action	Message
0:00-0:30	Problem + architecture	A score alone is not enough; RiskSentinel connects evidence and gates AI with deterministic policy.
0:30-1:15	Overview + system health	Show services and clean transaction-ops UI.
1:15-2:00	Normal transaction	Real ML/graph/agent/policy path, low-risk decision.
2:00-3:20	Suspicious collusion simulation	Show linked entities, graph signals, evidence and policy.
3:20-4:10	Safety case	Agent says BLOCK but weak graph -> policy REVIEW.
4:10-4:50	LLM outage	Agent degrades; deterministic pipeline survives.
4:50-5:30	Audit trace	Show tools, evidence, versions/reasons and final decision.
5:30-6:00	Metrics + limitations	Separate IEEE-CIS ML metrics from synthetic graph metrics; state limitations plainly.

21.2 Demo Preparation

- Use synthetic/demo transaction IDs only.
- Pre-generate graph data with a known seed.
- Do not hardcode demo outcomes; use fixtures that naturally exercise frozen rules.
- Keep a degraded-provider path available if external LLM connectivity fails.
- Never show old STALE_ metrics.

21.3 Five-Minute Submission Pitch Variant

Time	Content
0:00-0:25	Problem: a risk score is not an investigation.
0:25-0:55	Normal transaction and deterministic low-risk path; show agent gating if skipped.
0:55-2:10	Run Simulate Suspicious Collusion Pattern; show model-compatible fixtures, graph context and live case state.
2:10-3:05	Show bounded tool calls, evidence, reason codes and advisory recommendation.
3:05-3:35	Policy-safety fixture: agent says BLOCK but policy chooses REVIEW when corroboration is insufficient.
3:35-4:15	Show real held-out ML metrics and separate synthetic-graph metrics.
4:15-4:40	Open one audit trail and point to component/rule versions.
4:40-5:00	Close: Models detect. Graphs connect. Agents investigate. Policies decide.

22. Limitations, Claims & Ethics

The project becomes stronger when limitations are explicit.

22.1 Required Limitations

- IEEE-CIS is a public benchmark and does not represent Razorpay production distribution.
- The graph benchmark is synthetic; planted clusters test algorithms but do not establish real-world fraud-ring performance.
- NetworkX is prototype-scale and does not prove large payment-network throughput.
- LLM outputs vary by provider/model and are bounded rather than treated as ground truth.
- Human review/governance features beyond the recovered MVP may remain future work.
- Anonymized IEEE-CIS features limit semantic explanations.
- Illustrative FP/FN costs are not Razorpay economics.

22.2 Safe Public Claims

Claim	Allowed?	Condition
Uses public IEEE-CIS data for tabular fraud-risk research	Yes	Only after actual dataset/training pipeline is executed as stated.
Uses graph intelligence	Yes	Only after real synthetic graph generation/detection runs.
LLM makes final decisions	No	Policy is authoritative.
Works at Razorpay scale	No	Not measured.
Uses Razorpay data	No	False.
Evaluated on held-out test data	Yes	Only after Recovery Step 7 generates metrics from frozen artifacts.
Production-ready fraud platform	No	Prototype only.

22.3 Defense-Only Statement

RiskSentinel X is intended for defensive detection, investigation and model-governance research. Synthetic suspicious patterns are generated to test detection robustness, not to provide instructions for evading fraud controls.

23. Release Criteria & Definition of Done

What must be true before calling the MVP complete.

- ☐ Database tests execute successfully on a real Python environment.
- ☐ PostgreSQL container starts and /health verifies the database.
- ☐ IEEE-CIS training data is acquired legitimately and kept outside Git.
- ☐ ML training executes and produces real model/preprocessor/feature/threshold artifacts.
- ☐ Validation metrics are calculated from predictions and recorded.
- ☐ Synthetic graph data is generated and graph evaluation executes.
- ☐ Ground-truth isolation tests pass.
- ☐ Real LLM provider path works when configured; mock provider is test-only.
- ☐ Agent evidence validation and provider-outage tests pass.
- ☐ Policy no longer receives hash/random/mock scores.
- ☐ Agent BLOCK + weak graph does not auto-BLOCK.
- ☐ One real E2E transaction traverses ML -> Graph -> Agent -> Policy -> Audit.
- ☐ Dashboard reads persisted backend data; normal runtime has no MOCK_TRANSACTIONS.
- ☐ Held-out evaluation computes official metrics from frozen model artifacts.
- ☐ Suspicious-collusion simulator uses the real pipeline and model-compatible feature fixtures; it never injects risk scores/decisions.
- ☐ Normal graph API never exposes planted ground-truth cluster IDs or labels.
- ☐ Deterministic investigation gate can skip the LLM only when both ML and graph context are low according to versioned rules.
- ☐ No hardcoded official metrics remain.
- ☐ Docker/clean-start instructions are actually validated.
- ☐ No secrets are committed.
- ☐ No P0/P1 defects remain for the demo path.

Final MVP status string

Only after all critical gates pass should the project report: GENUINE RISKSSENTINEL X MVP COMPLETE.

24. Appendices

Repository structure, sample contracts, execution commands and references.

24.1 Recommended Repository Structure

```
risksentinel-x/
├── backend/
│   └── app/
│       ├── api/
│       ├── db/
│       ├── models/
│       ├── risk/
│       ├── graph/
│       ├── agent/
│       ├── policy/
│       └── audit/
├── frontend/
├── data/
│   ├── raw/
│   ├── processed/
│   └── synthetic/
├── models/
├── evaluation/
├── simulation/
├── tests/
├── docs/
├── docker-compose.yml
├── .env.example
├── BUILD_LOG.md
└── README.md
```

24.2 Example End-to-End Result Contract

```
{
  "transaction_id": "tx_demo_1052",
  "ml": {"risk_score": 0.48, "model_version": "xgb-ieeeecis-v1"},
  "graph": {"graph_risk": 0.11, "reason_codes": []},
  "agent": {
    "status": "COMPLETED",
    "recommendation": "BLOCK",
    "confidence": 0.95,
    "reason_codes": ["..."],
    "evidence": ["..."]
  },
  "policy": {
    "decision": "REVIEW",
    "policy_version": "policy-v1",
    "triggered_rules": ["POL-REVIEW-..."]
  }
}
```

The numbers above are an illustrative schema example, not benchmark results. The important behavior is that the agent can recommend BLOCK while deterministic policy still selects REVIEW when corroboration is insufficient.

24.3 Suggested Local Commands

```
# Backend tests
python -m pytest backend/tests

# Train ML after IEEE-CIS files are present
```

```
python -m evaluation.train_model

# Generate/evaluate graph (exact commands depend on final implementation)
python -m simulation.generate_graph --seed 42
python -m evaluation.evaluate_graph

# Docker
docker compose config
docker compose up --build

# Frontend (run only scripts present in package.json)
npm run lint
npm run typecheck
npm run build
```

24.4 Evidence Checklist for Every Numeric Claim

- Metric name and exact definition
- Dataset / split version
- Model or graph version
- Threshold source
- Evaluation script path
- Machine-readable result artifact
- Date/commit where practical

24.5 References

Source	URL	Use
Razorpay Docs - About Dashboard	https://razorpay.com/docs/payments/dashboard/	Used to ground dashboard patterns: analytics, recent activity, payments overview and operational navigation.
Razorpay Docs - Payments Dashboard Actions	https://razorpay.com/docs/payments/payments/dashboard/	Used to ground transaction list/detail drill-down behavior.
Razorpay Docs - Razorpay Home	https://razorpay.com/docs/payments/dashboard/razorpay-home/	Used to ground consolidated operational overview, quick actions and payment-insight patterns.
Kaggle - IEEE-CIS Fraud Detection Data	https://www.kaggle.com/c/ieee-fraud-detection/data	Official benchmark data description, file names, join key, target and competition-rule licensing.
Kaggle - IEEE-CIS data discussion by competition host	https://www.kaggle.com/c/ieee-fraud-detection/discussion/101203	Additional feature descriptions and context for TransactionDT / transaction fields.
Razorpay Docs - Track Payments	https://razorpay.com/docs/payments/mobile-app/track-manage/	Used to confirm date/status filters and transaction-detail interaction patterns.
OpenRouter - Stealth provider / Ox Alpha	https://openrouter.ai/provider/stealth	Current preview status, context window and provider caveats for the optional hosted development provider.
OpenRouter - Tool Calling Models	https://openrouter.ai/collections/tool-calling-models	Used only to verify that Ox Alpha appears in OpenRouter's current tool-calling collection; no latency/error-rate claim is inferred.

24.6 Final Working Principle

Build truth before breadth

Every critical claim must trace to executable code, real test data, persisted evidence and reproducible evaluation.

25. Companion PDF Review & Incorporated Enhancements

What was retained, corrected and added after reviewing the 20-page companion project documentation.

25.1 Strong Material Retained

- Clear problem framing: risk score alone is insufficient for investigation.
- Exactly two bounded agent tools for the MVP.
- Validation-derived policy thresholds and explicit false-positive / false-negative tradeoffs.
- The 15-case acceptance checklist covering malformed input, duplicate IDs, policy override, provider failure and audit persistence.
- Razorpay-inspired operations UX: compact transaction table, status hierarchy, drill-down investigation view and a defensive simulation action.
- A concise five-minute judging narrative and explicit honesty/limitations language.

25.2 Corrections Applied

Companion PDF Item	Why It Needed Correction	Master Specification Fix
Graph API example returned planted cluster_id such as SC-03	Ground-truth leakage risk; detector would expose the benchmark answer.	Runtime returns detected community_id only; planted IDs stay evaluation-only.
Suspicious-neighbour ratio listed without definition	Could accidentally mean neighbors with planted suspicious labels.	Omit or derive only from detector-produced structural risk, never ground truth.
Simple /score payload with only transaction ID, amount and customer	Insufficient for a real IEEE-CIS model with a large persisted feature schema.	Require a model-compatible feature dictionary or explicit adapter contract.
Fixed simulation said it should reliably trigger ML -> graph -> policy	Risk of hardcoding outcomes or forcing scores.	Fixtures are model-compatible and versioned; services compute every score/decision.
Ox Alpha provider section contained precise latency/error and model-family speculation	Those numbers/attribution were not traceable to the current official model listing.	Keep only current verifiable preview facts and a provider abstraction; remove unsupported stats/speculation.
Low-risk path skipped agent but graph behavior was not explicit	Skipping all downstream context can miss relationship risk.	Run ML + graph context first; deterministic gate skips only the LLM when both are low.

25.3 New Operational Enhancements Added

- Deterministic InvestigationGate with auditable SKIP_AGENT / RUN_AGENT result.
- Case lifecycle states for real-time UI and reliable SSE/WebSocket recovery.
- Simulation session IDs, deterministic seeds and strict separation from benchmark metrics.
- Read-only Policy & Agent Rules view for judges/debugging without editable production-like thresholds.
- Razorpay-aligned interaction patterns: date/status filters, recent activity, time-range selection and focused transaction detail drawer.
- Explicit detector-community-vs-ground-truth identity separation.
- Expanded 15-case acceptance matrix and five-minute pitch variant.

Scope discipline

These improvements strengthen the real MVP and its credibility. They do not reopen the deferred v2 research roadmap; calibration, counterfactuals, drift and retraining remain behind the genuine-MVP completion gate.